



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN

**THIẾT BỊ ĐEO THEO DÕI SỨC KHỎE, PHÁT HIỆN TẾ
NGÃ VÀ ĐỊNH VỊ DÙNG ESP32-S3**

BÁO CÁO CÔNG VIỆC CÁ NHÂN

Người thực hiện	Quách Gia Thịnh
Mã số sinh viên	24207105
Nhóm	03 thành viên
Phạm vi báo cáo	Phát hiện té ngã, đếm bước, định vị GPS và luồng cảnh báo

Phạm Hùng Tiến – Hồ Sĩ Phú – Quách Gia Thịnh

Mã nguồn đồ án: github.com/PhamHungTien/esp32-s3-health-monitor

Thành phố Hồ Chí Minh, Tháng 8 năm 2026

Nội dung phụ trách

Trong đồ án, em phụ trách IMU, đếm bước, phát hiện té ngã, GPS/NMEA và hợp đồng dữ liệu web cho chuyển động, cảnh báo, vị trí cùng hai thao tác điều khiển liên quan.

1. Mục tiêu công việc

Chức năng té ngã cần phân biệt va chạm thông thường với chuỗi rơi–va đập–bất động. Phần GPS phải nhận liên tục dữ liệu NMEA, kiểm tra checksum, phân biệt các trạng thái vận hành và công bố dữ liệu nhất quán cho OLED, Serial và dashboard web.

2. Mô-đun IMU, đếm bước và té ngã

2.1. Khởi tạo và chuẩn hóa

Thiết bị I2C tại 0x68 trả về WHO_AM_I=0x74. Em đọc sáu byte dữ liệu của các trục x, y, z , đổi sang đơn vị g và tính độ lớn:

Thanh ghi	Giá trị	Mục đích
0x75	chỉ đọc	Kiểm tra mã WHO_AM_I
0x6B	0x00	Đưa IMU ra khỏi chế độ ngủ
0x1C	0x00	Chọn thang gia tốc $\pm 2g$
0x1A	0x03	Bật bộ lọc số mức 3
0x3B	đọc 6 byte	Lấy liên tiếp a_x, a_y, a_z

$$A = \sqrt{a_x^2 + a_y^2 + a_z^2}.$$

Ở trạng thái đặt yên, phép đo thực tế đạt khoảng 1,07–1,08 g , phù hợp với gia tốc trọng trường và được dùng làm mốc bất động.

2.2. Máy trạng thái phát hiện té ngã

Em triển khai bốn trạng thái để kiểm tra đầy đủ diễn biến của một lần té ngã:

1. **NORMAL**: theo dõi bình thường;
2. **FREEFALL**: xác nhận gia tốc thấp trong một khoảng ngắn;
3. **IMPACT**: chờ va đập mạnh trong cửa sổ thời gian giới hạn;
4. **ALERT**: chỉ kích hoạt sau khi có giai đoạn ít chuyển động sau va đập.

Mọi trạng thái trung gian đều có timeout để quay về NORMAL nếu chuỗi sự kiện không hoàn chỉnh. Cấu trúc này loại nhiều báo giả do vùng tay hoặc đặt thiết bị mạnh xuống bàn.

```

1 // Bắt đầu nhánh xử lý khi máy trạng thái đang ở điều kiện vận động bình thường.
2 case FALL_NORMAL:
3 // Kiểm tra tổng gia tốc nhỏ hơn 0,45 g, dấu hiệu cơ thể có thể đang rơi tự do.
4 if (accelerationG < 0.45f) {
5 // Lưu mốc bắt đầu mức gia tốc thấp nếu đây là mẫu thấp đầu tiên.
6 if (lowGStartMs == 0) lowGStartMs = now;
7 // Yêu cầu điều kiện gia tốc thấp kéo dài ít nhất 120 ms để loại nhiễu tức thời.
8 if (now - lowGStartMs >= 120) {
9 // Chuyển máy trạng thái sang giai đoạn rơi tự do đã được xác nhận.
10 fallState = FALL_FREE_FALL;
11 // Lưu thời điểm chuyển trạng thái để kiểm soát thời hạn của giai đoạn tiếp theo.
12 fallStateMs = now;
13 // Kết thúc khối xác nhận đủ thời gian gia tốc thấp.
14 }
15 // Đi vào nhánh này khi tổng gia tốc đã trở lại từ 0,45 g trở lên.
16 } else {
17 // Xóa mốc gia tốc thấp vì chuỗi rơi tự do không còn liên tục.
18 lowGStartMs = 0;
19 // Kết thúc phép kiểm tra rơi tự do.
20 }
21 // Kiểm tra xung va đập trực tiếp lớn hơn 2,8 g.
22 if (accelerationG > 2.8f) {
23 // Chuyển sang trạng thái va đập khi phát hiện xung mạnh.
24 fallState = FALL_IMPACT;
25 // Lưu thời điểm va đập để bắt đầu cửa sổ kiểm tra bất động.
26 fallStateMs = now;
27 // Kết thúc nhánh phát hiện va đập trực tiếp.
28 }
29 // Thoát khỏi nhánh trạng thái bình thường của câu lệnh switch.
30 break;
31 // Bắt đầu nhánh chờ va đập sau khi đã xác nhận rơi tự do.
32 case FALL_FREE_FALL:
33 // Xác nhận va đập nếu tổng gia tốc vượt 2,2 g trong cửa sổ rơi tự do.
34 if (accelerationG > 2.2f) {
35 // Chuyển sang giai đoạn kiểm tra bất động sau va đập.
36 fallState = FALL_IMPACT;
37 // Lưu thời điểm va đập để tính cửa sổ xác nhận tiếp theo.
38 fallStateMs = now;
39 // Hủy chuỗi nếu sau 1.200 ms vẫn không xuất hiện va đập đủ mạnh.
40 } else if (now - fallStateMs > 1200) {
41 // Trở về trạng thái bình thường vì sự kiện rơi tự do không hoàn chỉnh.
42 fallState = FALL_NORMAL;
43 // Xóa mốc gia tốc thấp để lần giám sát kế tiếp bắt đầu sạch.
44 lowGStartMs = 0;
45 // Kết thúc phép kiểm tra va đập sau rơi tự do.
46 }
47 // Thoát khỏi nhánh rơi tự do.
48 break;

```

```

49
50 // Bắt đầu nhánh xử lý sau khi đã phát hiện va đập.
51 case FALL_IMPACT:
52 // Yêu cầu đã qua 1.800 ms và mức chuyển động nhỏ hơn 0,08 g.
53 if (now - fallStateMs > 1800 && motionLevel < 0.08f &&
54 // Đồng thời yêu cầu tư thế tĩnh có tổng gia tốc gần 1 g, trong khoảng 0,72–1,28 g.
55 accelerationG > 0.72f && accelerationG < 1.28f) {
56 // Chuyển sang cảnh báo té ngã khi toàn bộ điều kiện bất động đều đúng.
57 fallState = FALL_ALERT;
58 // Lưu thời điểm phát cảnh báo để phục vụ hiển thị, còi và thao tác hủy.
59 fallStateMs = now;
60 // Ghi thông báo cảnh báo ra Serial khi cổng chẩn đoán đang được mở.
61 Serial.println("[ALERT] PHAT HIEN TE NGA - CAN KIEM TRA NGUOI DUNG");
62 // Nếu sau 5 giây vẫn không đủ điều kiện bất động thì xem va đập là sự kiện giả.
63 } else if (now - fallStateMs > 5000) {
64 // Đưa máy trạng thái về bình thường để tiếp tục giám sát.
65 fallState = FALL_NORMAL;
66 // Kết thúc nhánh hết thời hạn chờ sau va đập.
67 }
68 // Thoát khỏi nhánh trạng thái va đập.
69 break;
70
71 // Bắt đầu nhánh cảnh báo đã được xác nhận.
72 case FALL_ALERT:
73 // Giữ nguyên cảnh báo cho đến khi người dùng hủy từ dashboard hoặc khởi động lại.
74 break;

```

Đoạn mã 1: Các điều kiện quyết định trong máy trạng thái té ngã

2.3. Đếm bước

Độ lớn gia tốc được tách thành thành phần nền và thành phần chuyển động. Một đỉnh chỉ được xem là bước khi vượt ngưỡng, cách đỉnh trước ít nhất 280 ms và nằm trong chuỗi có chu kỳ không quá 1,8 giây. Bước đầu tiên chỉ được cộng sau khi xuất hiện đỉnh thứ hai, nhờ đó việc cảm thiết bị lên không làm tăng bộ đếm.

```

1 // Cập nhật thành phần trọng lực bằng bộ lọc thông thấp với hệ số 0,03.
2 gravity += 0.03f * (totalG - gravity);
3
4 // Loại thành phần trọng lực khỏi tổng gia tốc để thu phần gia tốc do bước chân.
5 float dynamicG = totalG - gravity;
6
7 // Lọc tiếp gia tốc động với hệ số 0,25 để giảm nhiễu cao tần.
8 dynamicFiltered += 0.25f * (dynamicG - dynamicFiltered);
9
10 // Dòng trống phân tách bước lọc tín hiệu với bước phát hiện một chu kỳ bước.
11
12 // Mở cờ chờ đỉnh dương sau khi tín hiệu đã đi qua đáy âm nhỏ hơn -0,05 g.
13 if (dynamicFiltered < -0.05f) peakArmed = true;
14
15 // Yêu cầu tín hiệu vượt đỉnh dương 0,11 g khi cờ chờ đã được mở.
16 if (peakArmed && dynamicFiltered > 0.11f &&
17     now - lastPeakMs > 280 && totalG < 1.8f) {
18     // Tính khoảng giữa hai đỉnh; đặt bằng 0 nếu chưa từng có đỉnh trước đó.
19     uint32_t interval = lastPeakMs == 0 ? 0 : now - lastPeakMs;
20     // Chấp nhận nhịp bước có chu kỳ từ 280 đến 1.800 ms.
21     if (interval >= 280 && interval <= 1800) {
22         // Kiểm tra chuỗi đi bộ đã được xác nhận từ một cặp đỉnh trước hay chưa.
23         if (walkingSequence) {
24             // Khi chuỗi đã hợp lệ, mỗi đỉnh mới làm tăng bộ đếm thêm một bước.
25             stepCount++;
26         } else {
27             // Đi vào nhánh khởi tạo khi vừa thu được cặp đỉnh hợp lệ đầu tiên.
28             walkingSequence = true;
29         }
30         // Kết thúc nhánh lựa chọn cách cộng bước.
31     }
32     // Đi vào nhánh này khi khoảng đỉnh nằm ngoài miền nhịp bước cho phép.
33     } else {
34         // Hủy trạng thái chuỗi đi bộ để lần kế tiếp phải xác nhận lại.
35         walkingSequence = false;
36     }
37     // Kết thúc kiểm tra khoảng thời gian giữa hai đỉnh.
38 }
39
40 // Lưu mốc đỉnh hiện tại cho phép tính khoảng đỉnh ở lần sau.
41 lastPeakMs = now;
42
43 // Đóng cờ chờ đỉnh, buộc tín hiệu phải qua một đáy âm mới trước bước tiếp theo.
44 peakArmed = false;
45
46 // Kết thúc điều kiện xác nhận đỉnh dương.
47 }

```

Đoạn mã 2: Lọc gia tốc động và xác nhận chuỗi bước

3. Mô-đun định vị GPS

3.1. UART và thu dữ liệu

GPS hoạt động ở 9600 baud trên một UART phần cứng riêng. Sau khi thay lại dây đúng chiều tín hiệu, bộ đếm đã nhận hơn 7.000 byte NMEA liên tục. Việc đọc UART nằm trong mọi vòng lặp để không tràn bộ đệm. Phần thu dòng, checksum và parser được viết trực tiếp, không dùng TinyGPS++ hoặc thư viện GPS bên ngoài.



Figure 1: Mô-đun GPS UART tham chiếu. Đồ án xác nhận khối định vị bằng dữ liệu NMEA 9600 baud; cần thay ảnh nếu nhận mô-đun thật khác. Nguồn: Waveshare.

3.2. Kiểm tra và giải mã NMEA

Em gom dữ liệu từ ký tự “\$” đến cuối dòng, tính XOR phần thân và so sánh với checksum sau dấu “*”. Chỉ câu đúng checksum mới được tách trường. Câu GGA cung cấp chất lượng fix, số vệ tinh và vị trí; câu RMC cho biết bản tin vị trí có hợp lệ hay không.

Vĩ độ/kinh độ từ dạng độ–phút được đổi sang độ thập phân:

$$d_{decimal} = d + \frac{m}{60},$$

và đổi dấu cho bán cầu Nam hoặc Tây. Vị trí chỉ được đánh dấu FIX khi câu hợp lệ báo chất lượng tốt, ký hiệu bán cầu đúng và tọa độ nằm trong miền cho phép. Khi bản tin mới báo mất fix, trạng thái chuyển sang NOFIX; tọa độ hợp lệ gần nhất vẫn được giữ riêng để dùng trong màn hình cảnh báo.

```

1 // Đổi một ký tự hexadecimal thành giá trị 0–15 để đọc hai chữ số checksum.
2 int8_t NMEA_HexNibble(char c) {
3     if (c >= '0' && c <= '9') return c - '0';
4     if (c >= 'A' && c <= 'F') return c - 'A' + 10;
5     if (c >= 'a' && c <= 'f') return c - 'a' + 10;
6
7     // Trả -1 nếu ký tự không thuộc miền hexadecimal hợp lệ.
8     return -1;
9 }
10
11 // Khai báo hàm kiểm tra checksum và nhận chuỗi NMEA dạng ký tự làm đầu vào.
12 bool NMEA_ChecksumOK(const char *sentence) {
13     // Loại con trỏ rỗng hoặc chuỗi không bắt đầu bằng ký tự đô-la theo chuẩn NMEA.
14     if (!sentence || sentence[0] != '$') return false;
15
16     // Tìm dấu sao, vị trí phân cách phần dữ liệu với hai chữ số checksum.
17     const char *star = strchr(sentence, '*');
18
19     // Loại câu không có dấu sao hoặc không đủ hai ký tự checksum phía sau.
20     if (!star || strlen(star) < 3) return false;
21
22     // Đổi chữ số hexadecimal thứ nhất sau dấu sao thành bốn bit cao.
23     int8_t high = NMEA_HexNibble(star[1]);
24
25     // Đổi chữ số hexadecimal thứ hai thành bốn bit thấp.
26     int8_t low = NMEA_HexNibble(star[2]);
27
28     // Loại câu có một trong hai chữ số checksum không hợp lệ.
29     if (high < 0 || low < 0) return false;
30
31     // Dòng trống phân tách bước đọc checksum nhận được với bước tự tính checksum.
32
33     // Khởi tạo biến checksum tính toán bằng 0.
34     uint8_t checksum = 0;
35
36     // Duyệt các ký tự sau dấu đô-la và dừng ngay trước dấu sao.
37     for (const char *p = sentence + 1; p < star; p++) {
38         // XOR từng byte vào biến tích lũy đúng theo quy tắc checksum NMEA.
39         checksum ^= (uint8_t)*p;
40     }
41
42     // Kết thúc vòng lặp sau khi đã xử lý toàn bộ phần thân câu.
43
44     // Ghép bốn bit cao và bốn bit thấp thành byte checksum nhận được.
45     uint8_t expected = (uint8_t)((high << 4) | low);
46
47     // Chỉ chấp nhận câu khi checksum tự tính bằng checksum đi kèm.
48     return checksum == expected;
49
50     // Kết thúc thân hàm.
51 }

```

Đoạn mã 3: Tính và kiểm tra checksum NMEA trước khi tách trường

3.3. Đổi định dạng và lưu vị trí hợp lệ

```

1 // Đổi chuỗi ddddmm.mmmm hoặc dddmm.mmmm của NMEA sang độ thập phân.
2 double NMEA_ToDegrees(const char *value, char hemisphere) {
3     if (!value || !*value) return 0.0;
4     double raw = atof(value);

```

```

5 // Phần nguyên trước hai chữ số phút là số độ; phần còn lại là số phút.
6 int degrees = (int)(raw / 100.0);
7 double minutes = raw - degrees * 100.0;
8 double result = degrees + minutes / 60.0;
9
10 // Vĩ độ Nam và kinh độ Tây mang dấu âm trong hệ tọa độ thập phân.
11 if (hemisphere == 'S' || hemisphere == 'W') result = -result;
12 return result;
13 }
14
15 // Chỉ lưu khi đủ hai chuỗi tọa độ và hai ký hiệu bán cầu hợp lệ.
16 bool GPS_StorePosition(const char *latitude, char northSouth,
17                       const char *longitude, char eastWest) {
18     if (!latitude || !longitude || !latitude[0] || !longitude[0]) return false;
19     if (northSouth != 'N' && northSouth != 'S') return false;
20     if (eastWest != 'E' && eastWest != 'W') return false;
21
22     // Đổi hai trường NMEA rồi loại tọa độ vượt giới hạn địa lý.
23     double parsedLatitude = NMEA_ToDegrees(latitude, northSouth);
24     double parsedLongitude = NMEA_ToDegrees(longitude, eastWest);
25     if (fabs(parsedLatitude) > 90.0 ||
26         fabs(parsedLongitude) > 180.0) return false;
27
28     // Công bố vị trí, đánh dấu đã từng có tọa độ và lưu thời điểm fix mới nhất.
29     gpsLatitude = parsedLatitude;
30     gpsLongitude = parsedLongitude;
31     gpsFix = true;
32     gpsPositionKnown = true;
33     lastGPSFixMs = millis();
34     return true;
35 }

```

Đoạn mã 4: Đổi tọa độ NMEA và lưu vị trí sau khi kiểm tra miền

3.4. Tách trường GGA và RMC

```

1 // Loại ngay câu sai checksum trước khi sao chép hoặc tách trường.
2 void GPS_ParseSentence(const char *sentence) {
3     if (!NMEA_ChecksumOK(sentence)) return;
4
5     // Sao chép tối đa 95 ký tự và luôn kết thúc chuỗi bằng ký tự rỗng.
6     char copy[96];
7     strncpy(copy, sentence, sizeof(copy));
8     copy[sizeof(copy) - 1] = '\0';
9
10    // Cắt phần checksum để dấu sao không nằm trong trường cuối.
11    char *star = strchr(copy, '*');
12    if (star) *star = '\0';
13
14    // Tự tách tối đa 16 trường bằng cách thay dấu phẩy thành ký tự kết thúc chuỗi.
15    char *fields[16] = {nullptr};
16    uint8_t count = 1;
17    fields[0] = copy;
18    for (char *p = copy; *p && count < 16; p++) {

```



```

18     if (*p == ',') {
19         *p = '\0';
20         fields[count++] = p + 1;
21     }
22 }
23 if (count == 0) return;
24
25 // Nhận loại câu bằng hậu tố GGA hoặc RMC để hỗ trợ nhiều tiền tố vệ tinh.
26 bool isGGA = strstr(fields[0], "GGA") != nullptr;
27 bool isRMC = strstr(fields[0], "RMC") != nullptr;
28
29 // GGA cung cấp chất lượng fix, số vệ tinh, vĩ độ và kinh độ.
30 if (isGGA && count > 7) {
31     uint8_t quality = (uint8_t)atoi(fields[6]);
32     gpsSatellites = (uint8_t)atoi(fields[7]);
33     if (quality == 0 ||
34         !GPS_StorePosition(fields[2], fields[3][0],
35                             fields[4], fields[5][0])) {
36         gpsFix = false;
37     }
38
39 // RMC chỉ được chấp nhận khi trường trạng thái bằng A và tọa độ hợp lệ.
40 } else if (isRMC && count > 6) {
41     if (fields[2][0] != 'A' ||
42         !GPS_StorePosition(fields[3], fields[4][0],
43                             fields[5], fields[6][0])) {
44         gpsFix = false;
45     }
46 }

```

Đoạn mã 5: Parser NMEA tự viết cho câu GGA và RMC

3.5. Thu UART và trạng thái vận hành

```

1 // Đọc hết byte UART đang chờ để bộ đệm không tràn khi vòng lặp xử lý mô-đun khác.
2 void GPS_Poll() {
3     while (gpsSerial.available() > 0) {
4         char c = (char)gpsSerial.read();
5         gpsByteCount++;
6         lastGPSByteMs = millis();
7
8 // Ký tự độ-lơ-luôn bắt đầu một câu mới và xóa phần câu dở trước đó.
9         if (c == '$') {
10             gpsWorkingLength = 0;
11             gpsWorking[gpsWorkingLength++] = c;
12
13 // Ký tự xuống dòng kết thúc câu; parser chỉ nhận bộ đệm không rỗng.
14         } else if (c == '\n') {
15             if (gpsWorkingLength > 0) {
16                 gpsWorking[gpsWorkingLength] = '\0';
17                 GPS_ParseSentence(gpsWorking);
18                 gpsWorkingLength = 0;
19             }
20
21 // Bỏ ký tự CR và chỉ nối byte khi câu đã bắt đầu, còn đủ chỗ cho ký tự kết thúc.

```

```

20     } else if (c != '\r' && gpsWorkingLength > 0 &&
21               gpsWorkingLength < sizeof(gpsWorking) - 1) {
22         gpsWorking[gpsWorkingLength++] = c;
23     }
24 }
25 }
26
27 // Phân biệt chưa có byte, mất luồng UART, có fix mới và đang nhận nhưng chưa fix.
28 const char *GPS_StatusText() {
29     if (gpsByteCount == 0) return "WAIT";
30     if (millis() - lastGPSByteMs > 3000) return "LOST";
31     if (gpsFix && millis() - lastGPSFixMs < 10000) return "FIX";
32     return "NOFIX";
33 }

```

Đoạn mã 6: Gom câu NMEA từ UART và phân loại trạng thái GPS

Các trạng thái xuất ra OLED, Serial và API có ý nghĩa:

- **WAIT:** chưa nhận byte GPS;
- **LOST:** từng nhận nhưng mất dữ liệu quá 3 giây;
- **NOFIX:** đang nhận NMEA nhưng chưa có vị trí;
- **FIX:** có tọa độ hợp lệ gần thời điểm hiện tại.

3.6. Dữ liệu IMU/GPS và thao tác trên dashboard

Nhóm trường do em phụ trách được chia rõ theo chức năng:

- chuyển động: steps, fall, acceleration, motion;
- trạng thái GPS: gpsState, satellites, gpsBytes;
- vị trí: positionKnown, latitude, longitude.

Giao diện chỉ tạo liên kết bản đồ khi đã có vị trí; trạng thái FIX hiện tại vẫn được tách khỏi tọa độ hợp lệ gần nhất.

Hai yêu cầu POST được kiểm tra theo đúng biến trạng thái của thuật toán. Đường dẫn /api/reset-steps đưa bộ đếm bước về 0; /api/reset-alert đưa máy trạng thái về NORMAL và tắt buzzer ở lần cập nhật kế tiếp. Các thao tác này chỉ hoạt động trong mạng cục bộ do ESP32-S3 phát.

4. Kết quả kiểm thử

Nội dung	Kết quả quan sát	Đánh giá
Định danh IMU	WHO_AM_I = 0x74	Đạt
Gia tốc nghi	1,07–1,08g	Đạt
Đếm bước khi đặt yên	Giữ nguyên 0, không tự tăng	Đạt
Đếm bước với dữ liệu tổng hợp	20 chu kỳ chuyển động cho kết quả 20 bước; nhiễu nhỏ và một rung đơn cho 0 bước	Đạt
Đếm bước khi đi bộ	Đã có thuật toán; chưa thu đủ số bước chuẩn để tính sai số	Cần thử thêm
UART GPS	Bộ đếm byte tăng liên tục	Đạt
GPS ngoài trời/gần cửa sổ	FIX, 4 vệ tinh ở lần chạy cuối, có tọa độ	Đạt
GPS khi tín hiệu kém	NOFIX, không tạo tọa độ mới	Đạt
Máy trạng thái với dữ liệu tổng hợp	Chuỗi rơi–va đập–bắt động vào ALERT; chuỗi thiếu va đập quay về NORMAL	Đạt
Chuỗi té ngã	Đã cài máy trạng thái; chưa thử ngã người thật vì yêu cầu an toàn	Cần thử thêm
Dashboard IMU/GPS	Bước, gia tốc, trạng thái té ngã, GPS và tọa độ cập nhật đúng trường	Đạt
Điều khiển web cục bộ	Đặt lại bước và hủy ALERT trả phản hồi JSON thành công	Đạt

Phép thử trên máy tính dùng lại đúng hệ số và ngưỡng của firmware trong tệp `tests/algorithm_selfcheck.py`. Kết quả này kiểm tra tính nhất quán của logic, không thay thế phép thử độ chính xác khi đeo thiết bị.

5. Kết quả đạt được

Các giá trị đầu ra

Phần chương trình cung cấp độ lớn gia tốc, số bước, trạng thái té ngã, bộ đếm byte GPS, số vệ tinh, vĩ độ, kinh độ và trạng thái WAIT/LOST/NOFIX/FIX cho cả OLED, Serial và API web. Nhật ký phần cứng lần chạy cuối ghi nhận GPS FIX với 4 vệ tinh và tọa độ khoảng 10,757° Bắc, 106,676° Đông.

6. Nhận xét và hướng hoàn thiện

IMU và GPS đã hoạt động; GPS có vị trí hợp lệ ở nơi thoáng. Đếm bước cần đổi chiều bằng quãng đi chuẩn. Phát hiện té ngã cần thử an toàn với nhiều tư thế và bổ sung nút hủy cảnh báo.